Expo-Einstieg und Annahmen-Map

Vom Key Screen zur ersten Route

Beschreibung

Andrea Muster leitet das Zooh!-Projektteam. Die User Journey mit den Key Screens steht auf Papier; jetzt beginnt der Klickprototyp. Bevor die Gruppe Code schreibt, überarbeitet sie ihre Annahmen-Map: Welche Annahmen hinter den Key Screens sind wichtig und unsicher, und woran würde man merken, dass sie falsch sind? Danach legen Sie mit React Native und Expo ein Projekt an und bauen jeden Key Screen als eigene Route. Pflichtteile ca. 50 Minuten; Sie arbeiten in Ihrer Projektgruppe in Miro und auf dem Schulnotebook.

Inhaltsverzeichnis

1. EINSTIEG UND THEORIE 2

1.1 AUSGANGSLAGE UND ZIELE 2
1.2 THEORIE: ANNAHMEN-MAP 2
1.3 THEORIE: LOW-FI-KLICKPROTOTYP MIT EXPO 3

2. AUFGABE ABLAUF 6

2.1 EINFÜHRUNG 6
2.2 TEILAUFGABE A: ANNAHMEN-MAP VERSION 2 6
2.3 TEILAUFGABE B1: PROJEKT ANLEGEN UND STARTEN 6
2.4 TEILAUFGABE B2: KEY SCREENS ALS ROUTEN 6
2.5 TEILAUFGABE B3: STARTSCREEN UND TEST 7
2.6 HAUSAUFGABE UND ZUSATZ 7

3. ERGEBNISPRÜFUNG UND ABGABE 7

3.1 CHECKLISTE 7
3.2 CHECKPOINT-FRAGEN 7

1. Einstieg und Theorie

1.1 Ausgangslage und Ziele

Der Klickprototyp ist ab heute eine App: Jeder Key Screen aus Aufgabe 5.1 wird eine Datei, und die Journey wird zu Links zwischen diesen Dateien. Das Ergebnis ist Iteration 1 Ihres Prototyps (40 % der Modulnote) und die Grundlage für die Tests in Block 7. Die Annahmen-Map Version 2 gehört zur Projektdokumentation (Assumption Map).
Nach dieser Aufgabe können Sie:

- Annahmen zu Key Screens und User Journey sammeln, nach Wichtigkeit und Sicherheit einordnen und die drei wichtigsten unsicheren Annahmen als Hypothese mit Prüfkriterium formulieren (Kompetenzband A2 «Benutzerprofile und Anforderungsspezifikation», Niveau intermediate).
- ein Expo-Projekt anlegen, starten und auf dem Smartphone oder im Browser anzeigen (C1 «Controls, Widgets und Prototyp», Niveau beginner).
- jeden Key Screen als Route mit Titel, Platzhaltern und Stichworten anlegen und vom Startscreen aus verlinken (B2 «Benutzerschnittstelle entwerfen», Niveau advanced, Einstieg Klickprototyp).

Teilaufgabe	**Zeit**	**Produkt**
A Annahmen-Map Version 2	15 min	Map mit 3 Hypothesen auf dem Board
B1 Projekt anlegen und starten	12 min	App läuft in Expo Go oder im Browser
B2 Key Screens als Routen	18 min	eine Datei pro Key Screen, Titel im Stack
B3 Startscreen und Test	5 min	Startscreen mit Link zu jedem Screen
Zusatz (advanced)	offen	Testszenario pro Hypothese

Vorwissen und Umgebung: Key-Screen-Tabelle und User Journey aus 5.1, Annahmen-Map aus dem Scoping (Version 1; fehlt sie, beginnen Sie neu), Gruppen-Board in Miro, Schulnotebook mit Node.js LTS, Smartphone mit Expo Go (App Store oder Google Play).

1.2 Theorie: Annahmen-Map

Jedes Konzept stützt sich auf Annahmen: «Besucher kaufen Tickets lieber in der App als an der Kasse.» Die Annahmen-Map (Steimle & Wallach, S. 134–141) macht sie sichtbar und zeigt, welche Sie zuerst prüfen müssen. Sie ordnen jede Annahme in zwei Richtungen ein:

- **Wichtigkeit:** Was passiert mit dem Konzept, wenn die Annahme falsch ist? Wenn der Hauptflow scheitert, ist sie wichtig.
- **Sicherheit:** Wie gut wissen Sie es? Beobachtung und Tests machen sicher, Bauchgefühl nicht.

	sicher	**unsicher**
wichtig	Fundament, nicht testen	**zuerst prüfen**
unwichtig	ignorieren	später, falls Zeit

Aus den wichtigen und unsicheren Annahmen werden **Hypothesen** mit einem **Prüfkriterium**. Das Kriterium legt fest, woran Sie merken, dass die Annahme stimmt, bevor Sie testen. Es bereitet die Validierungsplanung vor (S. 142–148), aus der in Block 7 die Testaufgaben entstehen.

```
Annahme:       Besucher zeigen das QR-Ticket am Drehkreuz ohne Hilfe.
Wichtig?       hoch (ohne Ticket kein Eintritt)
Sicher?        tief (noch nie getestet)
Hypothese:     Wir glauben, dass Besucher das QR-Ticket allein finden
               und vorzeigen können.
Prüfkriterium: Mindestens 4 von 5 Testpersonen schaffen es beim
               ersten Versuch ohne Hinweis.
```

Hinweis: Annahmen sind keine Lösungen. «Die App braucht einen grossen Button» ist eine Idee; «Besucher finden den Button ohne Erklärung» ist eine Annahme, die man prüfen kann.

1.3 Theorie: Low-Fi-Klickprototyp mit Expo

Ein **Low-Fi-Klickprototyp** zeigt Struktur und Ablauf, nicht Gestaltung: graue Platzhalter statt Bilder, Stichworte statt fertiger Texte. Die Fidelity bleibt bewusst niedrig, solange Fragen zum Flow offen sind. Sonst kommentieren Testpersonen später Farben statt Abläufe. Visuelles Design und Code-Qualität werden nicht bewertet.

Mit **expo-router** ist jede Datei in `src/app/` eine Route. Der Dateiname ist der Pfad; `_layout.tsx` legt fest, wie die Screens zusammenhängen.

Datei	Route	Aufgabe
`src/app/_layout.tsx`	-	`Stack` mit Titel pro Screen
`src/app/index.tsx`	`/`	Startscreen
`src/app/lageplan.tsx`	`/lageplan`	ein Key Screen

Regel: Dateinamen in kebab-case ohne Umlaute und Sonderzeichen (`fuetterung-detail.tsx`). Text steht nur in `<Text>`. Styles mit `StyleSheet.create`. Tippbare Elemente mit `Pressable`, nicht `TouchableOpacity`.

Die gemeinsame Basis besteht aus zwei kleinen Dateien. Legen Sie zuerst Farben und Abstände in `src/theme.ts` an, damit alle Screens gleich aussehen:

```
export const farben = {
  hintergrund: "#FFFFFF",
  text: "#1A1A1A",
  textSekundaer: "#555555",
  rahmen: "#9E9E9E",
  flaeche: "#EEEEEE",
  primaer: "#1B5E20",
  primaerText: "#FFFFFF",
  fehler: "#B71C1C",
};

export const abstand = { s: 8, m: 16, l: 24 };
```

Danach die Komponente `Platzhalter` in `src/components/platzhalter.tsx`, ein grauer Kasten mit Beschriftung für Bild, Karte oder Liste:

```
import { StyleSheet, Text, View } from "react-native";
import { abstand, farben } from "@/theme";

type Props = { label: string; hoehe?: number };

export function Platzhalter({ label, hoehe = 120 }: Props) {
  return (
    <View style={[styles.box, { height: hoehe }]} accessibilityLabel={label}>
      <Text style={styles.label}>{label}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  box: {
    backgroundColor: farben.flaeche,
```

```
    borderWidth: 1,
    borderColor: farben.rahmen,
    borderStyle: "dashed",
    borderRadius: 8,
    alignItems: "center",
    justifyContent: "center",
    padding: abstand.m,
  },
  label: { color: farben.textSekundaer },
});
```

Ein Key Screen sieht dann so aus. Das Beispiel zeigt den Screen «Lageplan» in `src/app/lageplan.tsx`; Ihre Screens heissen so, wie Ihre Key Screens in 5.1:

```
import { ScrollView, StyleSheet, Text } from "react-native";
import { Platzhalter } from "@/components/platzhalter";
import { abstand, farben } from "@/theme";

export default function Lageplan() {
  return (
    <ScrollView
      contentInsetAdjustmentBehavior="automatic"
      contentContainerStyle={styles.inhalt}
    >
      <Platzhalter label="Karte des Zoos mit Standort" hoehe={240} />
      <Text style={styles.text}>Zweck: Weg zur Fütterung finden</Text>
      <Text style={styles.text}>Hauptaktion: «Route starten»</Text>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  inhalt: { padding: abstand.m, gap: abstand.m },
  text: { fontSize: 16, color: farben.text },
});
```

Der Titel in der Kopfzeile kommt aus `src/app/_layout.tsx`:

```
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "Zooh!" }} />
      <Stack.Screen name="lageplan" options={{ title: "Lageplan" }} />
      <Stack.Screen name="fuetterung" options={{ title: "Fütterung" }} />
    </Stack>
  );
}
```

Der Weg zwischen den Screens entsteht durch `Link` im Startscreen `src/app/index.tsx`:

```
import { Link } from "expo-router";
import { StyleSheet, Text, View } from "react-native";
import { abstand, farben } from "@/theme";

export default function Start() {
  return (
    <View style={styles.inhalt}>
      <Text style={styles.titel}>Zooh!</Text>
```

```
      <Link href="/lageplan" style={styles.link}>
        Lageplan
      </Link>
      <Link href="/fuetterung" style={styles.link}>
        Fütterung
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  inhalt: { padding: abstand.m, gap: abstand.m },
  titel: { fontSize: 24, fontWeight: "700", color: farben.text },
  link: { fontSize: 18, color: farben.primaer, paddingVertical: abstand.s },
});
```

Weiterführend: Expo Router Einführung.

2. Aufgabe Ablauf

2.1 Einführung

Sie arbeiten in Ihrer festen Projektgruppe (3–4 Personen). Pflichtteile: **50 Minuten** (A 15 + B1 12 + B2 18 + B3 5).

Rolle	Aufgabe
Build-Verantwortliche(r)	legt das Projekt an, startet den Server, hilft bei Fehlern
Screens (zwei Personen)	teilen die Key Screens auf und schreiben die Dateien
Review (bei 4 Personen)	prüft Dateinamen und Regeln, pflegt die Annahmen-Map

2.2 Teilaufgabe A: Annahmen-Map Version 2

Zeit: ca. 15 min, Gruppe, Miro

1. Öffnen Sie die Annahmen-Map aus dem Scoping auf dem Gruppen-Board und duplizieren Sie den Frame als «Annahmen-Map V2».
2. Sammeln Sie 5 Minuten still neue Annahmen aus Key Screens und Journey (je eine Karte). Beispiel: «Die Persona gibt ihre E-Mail-Adresse für das Ticket an.»
3. Ordnen Sie alle Karten in die Matrix Wichtigkeit × Sicherheit ein. Verschieben Sie alte Karten, wenn Sie inzwischen mehr wissen.
4. Wählen Sie die drei wichtigsten unsicheren Annahmen und tragen Sie sie in die Tabelle ein:

Annahme	wichtig?	sicher?	Hypothese	Prüfkriterium
…	hoch	tief	Wir glauben, dass …	Mind. x von 5 …

Hypothese: «Wir glauben, dass …»; Prüfkriterium: zählbar (Personen, Zeit, Fehlversuche).

→ Produkt: Annahmen-Map V2 mit Matrix und Tabelle der drei Hypothesen mit Prüfkriterium auf dem Gruppen-Board

2.3 Teilaufgabe B1: Projekt anlegen und starten

Zeit: ca. 12 min, Build-Verantwortliche(r) führt, Gruppe schaut zu

1. Terminal in einem Ordner **ausserhalb** von OneDrive öffnen, Projekt anlegen:

```
npx create-expo-app@latest zooh
cd zooh
npm run reset-project
```

2. Frage nach `app-example`: Antworten Sie `n`.
3. Starten Sie `npx expo start` und scannen Sie den QR-Code mit Expo Go.
4. Mit `w` öffnet sich die App im Browser (Plan B).

Achtung: Gleiches WLAN nötig; sonst `npx expo start --tunnel`. Ohne Node.js (Plan C): Expo Snack im Browser.

→ Produkt: Die App läuft auf dem Smartphone oder im Browser; Screenshot im Board

2.4 Teilaufgabe B2: Key Screens als Routen

Zeit: ca. 18 min, Gruppe in Rollen, Notebook

1. Legen Sie `src/theme.ts` und `src/components/platzhalter.tsx` an (Code oben, abtippen oder kopieren).
2. Erstellen Sie pro Key Screen aus Ihrer Tabelle in 5.1 eine Datei in `src/app/`, z. B. `tickets.tsx`. Teilen Sie die Screens unter den zwei Screen-Schreibenden auf.

3. Jeder Screen enthält: den Titel oder Zweck als `<Text>`, einen oder zwei `Platzhalter` für Bild, Karte oder Liste (mit Beschriftung) und zwei bis drei Stichworte (Zweck, Hauptaktion aus Ihrer Tabelle).
4. Tragen Sie jeden Screen in `src/app/_layout.tsx` als `Stack.Screen` mit Titel ein.

Die Stichworte dürfen unvollständig bleiben. Hauptsache, jeder Screen ist erreichbar und erkennbar.

→ Prüfen Sie jeden Dateinamen: kebab-case, keine Umlaute, Endung `.tsx`, `export default` vorhanden.

→ Produkt: eine Datei pro Key Screen in `src/app/`, alle in `_layout.tsx` mit Titel eingetragen

2.5 Teilaufgabe B3: Startscreen und Test

Zeit: ca. 5 min, Gruppe

1. Ersetzen Sie `src/app/index.tsx` durch einen Startscreen mit Titel und einem `Link` zu jedem Key Screen (Beispiel oben).
2. Tippen Sie auf dem Smartphone jeden Link an und gehen Sie über die Kopfzeile zurück. Öffnet sich ein Screen nicht, fehlt meist der `export default` oder der Dateiname weicht vom `href` ab.
3. Lesen Sie die Meldung im Terminal oder die rote Fehlerseite: Die erste Zeile nennt Datei und Ursache.

→ Produkt: Startscreen mit funktionierenden Links zu allen Key Screens; Screenshot des Startscreens

2.6 Hausaufgabe und Zusatz

Hausaufgabe (nicht in der Pflichtzeit, ca. 20 min): Schliessen Sie fehlende Screens ab, erstellen Sie das ZIP ohne `node_modules` und geben Sie es mit dem Link zur Annahmen-Map V2 ab.

Zusatz für schnelle Gruppen (advanced): Formulieren Sie für jede Hypothese ein Testszenario als Aufgabe an die Testperson («Zeigen Sie dem Drehkreuz Ihr Ticket»), nicht als Klickanleitung. Notieren Sie im Code als Kommentar, welcher Screen zu welcher Hypothese gehört (`// H1: Ticket finden`).

3. Ergebnisprüfung und Abgabe

3.1 Checkliste

Prüfen Sie vor der Abgabe:

- Die Annahmen-Map V2 zeigt alle Karten in der Matrix und drei Hypothesen mit zählbarem Prüfkriterium.
- Das Projekt startet mit `npx expo start`, und die App läuft in Expo Go oder im Browser.
- Jeder Key Screen aus 5.1 ist eine Datei in `src/app/` (kebab-case, keine Umlaute) mit Titel im `Stack`.
- Jeder Screen hat Titel, mindestens einen `Platzhalter` und Stichworte; Text steht nur in `<Text>`.
- Der Startscreen verlinkt jeden Key Screen mit `Link`.
- Keine Namen von Mitlernenden oder Testpersonen im Projekt oder auf dem Board.

3.2 Checkpoint-Fragen

Beantworten Sie die Fragen zu zweit; die Lehrperson fragt stichprobenartig.

1. Wie ordnen Sie eine Annahme in die Annahmen-Map ein, und welche prüfen Sie zuerst?
2. Was macht aus einer Annahme eine Hypothese mit Prüfkriterium?
3. Wie wird aus einer Datei in `src/app/` eine Route, und wozu dient `_layout.tsx`?
4. Warum bleibt der Klickprototyp bewusst Low-Fi?

→ Abgabe: ZIP des Projekts **ohne** `node_modules` (oder Link zum Git-Repository) und Link zur Annahmen-Map V2 (Leserecht für die Lehrperson) in der Teams-Aufgabe «6.1 Expo-Einstieg» bis So 01.11.2026. Eine Abgabe pro Gruppe. Die Lösungen werden nach der Abgabe freigeschaltet.